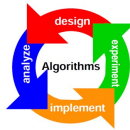


Estrategias algorítmicas

José A. Rodríguez Melquiades

5 de julio de 2026

Backtracking



1 Backtracking

2 n-Reinas

- Algoritmo basada en el uso de árbol

Backtracking

- Problemas del mundo real: Problemas NP.

Backtracking

- **Problemas del mundo real:** Problemas NP.
 - ▶ **Hallar:** Una solución a través de una busca exhaustiva, mediante un número finito de posibilidades.

Backtracking

- **Problemas del mundo real:** Problemas NP.
 - ▶ **Hallar:** Una solución a través de una busca exhaustiva, mediante un número finito de posibilidades.
- **Necesidad:** Desarrollar técnicas de búsqueda para reducir el espacio de soluciones a un tamaño más pequeño.

Backtracking

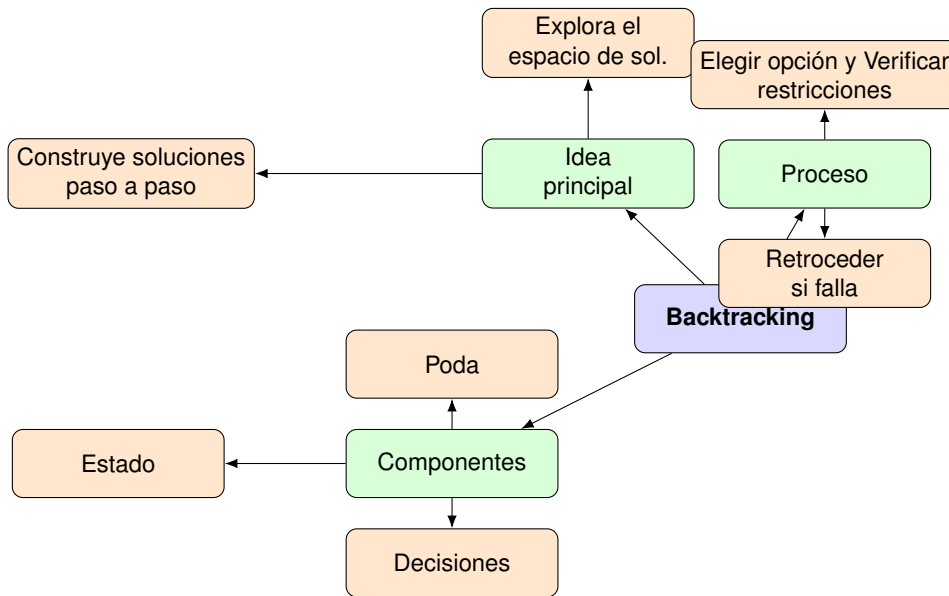
- **Problemas del mundo real:** Problemas NP.
 - ▶ **Hallar:** Una solución a través de una busca exhaustiva, mediante un número finito de posibilidades.
- **Necesidad:** Desarrollar técnicas de búsqueda para reducir el espacio de soluciones a un tamaño más pequeño.
- **Búsqueda exhaustiva:** Técnica general que en forma sistemática **busca soluciones en un espacio**. El proceso sucede en un árbol en el cual se representan las posibles soluciones.

- *Backtracking*: Técnica general descrito como una búsqueda exhaustiva que evita buscar todas las posibilidades.

- *Backtracking*: Técnica general descrito como una búsqueda exhaustiva que evita buscar todas las posibilidades.
 - ▶ Idea principal.

- *Backtracking*: Técnica general descrito como una búsqueda exhaustiva que evita buscar todas las posibilidades.
 - ▶ Idea principal.
 - ▶ Componentes.

- *Backtracking*: Técnica general descrito como una búsqueda exhaustiva que evita buscar todas las posibilidades.
 - ▶ Idea principal.
 - ▶ Componentes.
 - ▶ Proceso.



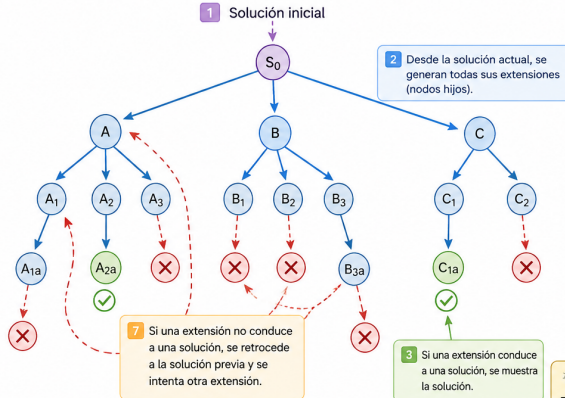
- **Idea:** Algoritmo recursivo que intenta obtener una solución por partes. Cuando llega a un "callejón sin salida" construye una solución o retrocede e intenta elegir diferentes valores para las partes. Finalmente **verifica** si la solución es válida en el nivel más profundo de la recursividad.

- **Idea:** Algoritmo recursivo que intenta obtener una solución por partes. Cuando llega a un "callejón sin salida" construye una solución o retrocede e intenta elegir diferentes valores para las partes. Finalmente **verifica** si la solución es válida en el nivel más profundo de la recursividad.
- **Algoritmo general:**
 - 1: Escoger una solución inicial.
 - 2: Explorar todas las extensiones de la solución actual.
 - 3: **if** Una extensión conduce a una solución **then**
 - 4: Mostrar la solución.
 - 5: **else**
 - 6: **if** Una extensión no conduce a una solución **then**
 - 7: Retroceder a la solución previa y ensayar una extensión diferente.
 - 8: **end if**
 - 9: **end if**
 - 10: **repeat**
 - 11: Repetir los pasos 2-9.
 - 12: **until** Que se hallan explorado todas las posibles soluciones

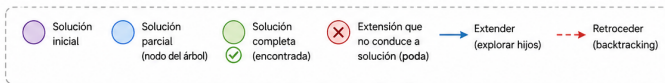
Estrategia Backtracking: funcionamiento

Algoritmo general:

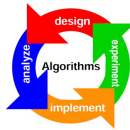
- 1 Escoger una solución inicial.
- 2 Explorar todas las extensiones de la solución actual.
- 3 **if** Una extensión conduce a una solución **then**
 - 4 Mostrar la solución.
- 5 **else**
 - 6 **if** Una extensión no conduce a una solución **then**
 - 7 Retroceder a la solución previa y ensayar una extensión diferente.
 - 8 **end if**
- 9 **end if**
- 10 **repeat**
 - Repetir los pasos 2-9.
- 12 **until** Que se hallan explorado todas las posibles soluciones



 **Idea clave:**
Explorar en profundidad el espacio de soluciones, retrocediendo cuando una opción no lleva a una solución válida, hasta haber explorado todas las posibles soluciones.



Backtracking



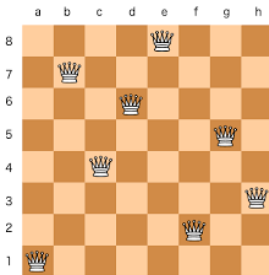
1 Backtracking

2 n-Reinas

- Algoritmo basada en el uso de árbol

n-Reinas

- **Objetivo:** Colocar n -reinas en un tablero de ajedrez de $n \times n$, de modo tal que dos reinas no se amenacen entre sí, es decir, no puede haber dos reinas en la misma fila, columna o diagonal.



Algoritmo basado en el uso de árbol

- *Backtracking* es una búsqueda en profundidad¹ de un árbol.

¹Depth-first search

Algoritmo basado en el uso de árbol

- *Backtracking* es una búsqueda en profundidad¹ de un árbol.
- Busca en profundidad:

¹Depth-first search

Algoritmo basado en el uso de árbol

- *Backtracking* es una búsqueda en profundidad¹ de un árbol.
- Busca en profundidad:
 - ▶ Se define para grafos, pero en general puede ser usado para árboles, ya que *backtracking* involucra solamente búsqueda en un árbol.

¹Depth-first search

- Busca en profundidad: (cont.)

- Busca en profundidad: (cont.)

- ▶ No requiere que el hijo sea visitado en cualquier orden, sin embargo el hijo será visitado desde un nodo de izquierda a derecha.

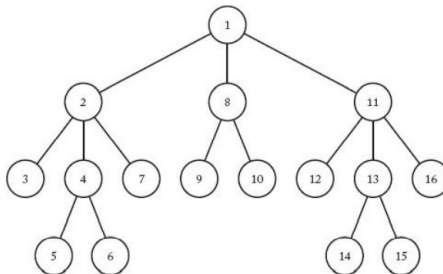
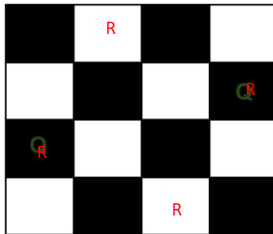


Figura 2: Orden de visita de los nodos del árbol.

- Ejemplo:

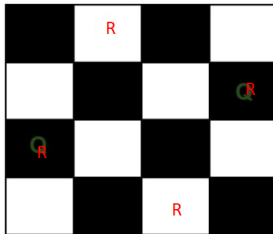
- Ejemplo:

- ▶ Actividad: Colocar cuatro reinas en un tablero de 4×4 .



- Ejemplo:

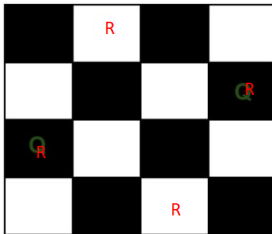
- ▶ Actividad: Colocar cuatro reinas en un tablero de 4×4 .



- ▶ Criterio: No colocar a dos reinas en la misma fila.

- Ejemplo:

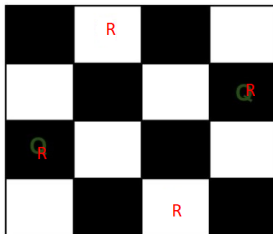
- ▶ Actividad: Colocar cuatro reinas en un tablero de 4×4 .



- ▶ Criterio: No colocar a dos reinas en la misma fila.
- ▶ Solución: La instancia puede ser solucionada asignando a cada reina una fila diferente y verificar con que combinaciones de columnas se obtiene la solución.

- Ejemplo:

- ▶ Actividad: Colocar cuatro reinas en un tablero de 4×4 .



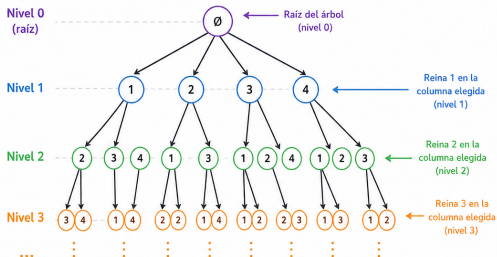
- ▶ Criterio: No colocar a dos reinas en la misma fila.
- ▶ Solución: La instancia puede ser solucionada asignando a cada reina una fila diferente y verificar con que combinaciones de columnas se obtiene la solución.
- ▶ Soluciones candidatas: Cada reina puede ser colocada en una de las cuatro columnas, entonces se tiene $4 \times 4 \times 4 \times 4 = 256$ soluciones candidatas.

- Ejemplo: (cont.)

- Ejemplo: (cont.)

- **Soluciones candidatas:** Construir un árbol en el cual la columna escogida para la reina 1 (en la columna 1) sea guardada en el nivel 1 del árbol. La columna escogida para la reina 2 (en la columna 2) será guardada en nivel 2 del árbol. Así sucesivamente. La raíz del árbol esta en el nivel 0.

Construir un árbol: En cada nivel del árbol se elige una columna (reina).
La columna elegida para la **reina 1 (columna 1)** se guarda en el nivel 1 del árbol.
La columna elegida para la **reina 2 (columna 2)** se guarda en el nivel 2 del árbol. Así sucesivamente.
La raíz del árbol está en el **nivel 0**.



Regla:

Cada nodo guarda la columna elegida para la reina de ese nivel del árbol.

Nivel del árbol	Reina colocada en la columna
0 (raíz)	—
1	Columna 1
2	Columna 2
3	Columna 3
...	...

Nodo: representa un estado parcial (elección de columnas hasta ese nivel). El número dentro del nodo es la columna elegida para la reina de ese nivel.

Arco: conecta un nodo padre con un nodo hijo.
Representa la decisión de la siguiente reina (siguiente nivel).

- Ejemplo: (cont.)

- **Ejemplo:** (cont.)

- ▶ **Camino:** Es una solución candidata cuyo recorrido será desde la raíz hasta la hoja. Este camino es parte del espacio de estados árbol. El árbol total tiene 256 hojas, uno para cada solución candidata. Una parte está dado por:

$[< 1, 1 >, < 2, 1 >, < 3, 1 >, < 4, 1 >]$

$[< 1, 1 >, < 2, 1 >, < 3, 1 >, < 4, 2 >]$

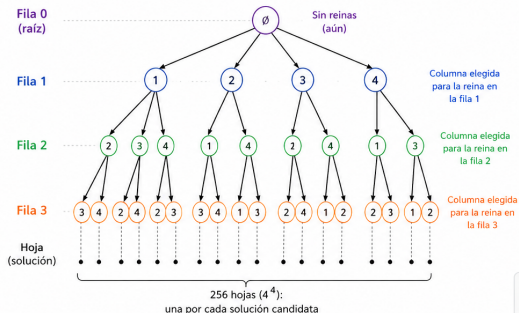
$[< 1, 1 >, < 2, 1 >, < 3, 1 >, < 4, 3 >]$

$[< 1, 1 >, < 2, 1 >, < 3, 1 >, < 4, 4 >]$

Cada par $< i, j >$ significa que la reina en la fila i es colocado en la columna j . Por lo tanto, los nodos son visitados de modo tal que los hijos de un nodo son visitados de izquierda a derecha.

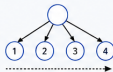
- Ejemplo: (cont.)

Camino: es una **solución candidata** cuyo recorrido será desde la raíz hasta la hoja. Cada camino corresponde a una colocación completa de las 4 reinas (una por fila). El árbol total tiene **256** hojas (4^4): una para cada solución candidata.



Orden de visita: izquierda a derecha

Los nodos se visitan de modo tal que los hijos de cada nodo se exploran de izquierda a derecha.



Ejemplos de caminos (soluciones candidatas)

camino = $\langle 1, 1 \rangle, \langle 2, 1 \rangle, \langle 3, 1 \rangle, \langle 4, 1 \rangle$

(columna de la fila 1) (columna de la fila 2) (columna de la fila 3) (columna de la fila 4)

camino = $\langle 1, 1 \rangle, \langle 2, 1 \rangle, \langle 3, 1 \rangle, \langle 4, 2 \rangle$

camino = $\langle 1, 1 \rangle, \langle 2, 1 \rangle, \langle 3, 1 \rangle, \langle 4, 3 \rangle$

camino = $\langle 1, 1 \rangle, \langle 2, 1 \rangle, \langle 3, 1 \rangle, \langle 4, 4 \rangle$

...

$\langle i, j \rangle$ significa que la reina en la fila i está colocada en la columna j .

Leyenda

○ **Nodo:** estado parcial (elección de columnas hasta esa fila). El número dentro del nodo indica la columna elegida para esa fila.

→ **Arco:** decisión de colocar la reina de la siguiente fila en una columna.

----- **Camino:** secuencia de decisiones desde la raíz hasta una hoja (solución candidata).

- Ejemplo: (cont.)

- **Ejemplo:** (cont.)

- ▶ La Fig. 3 muestra una parte del espacio de estados árbol, donde $\langle i, j \rangle$ en cada nodo significa que la reina en la fila i es colocado en la columna j .

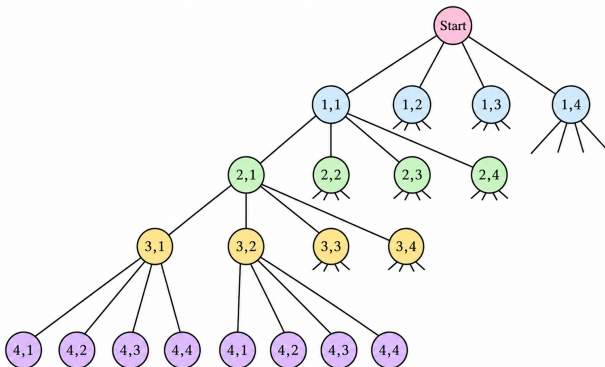
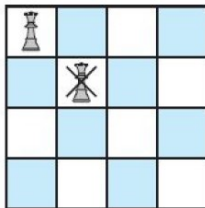
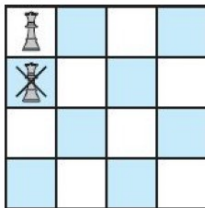


Figura 3: Parte del espacio de estados árbol.

- Ejemplo: (cont.)

- **Ejemplo:** (cont.)

- ▶ La sgte Fig. muestra que si la reina 1 es colocada en $\langle 1, 1 \rangle$, entonces la reina 2 no podrá ser colocado en la columna 1 o columna 2.



- Resumen:

● Resumen:

- ▶ *Backtracking* consiste en hacer una **busca** en profundidad en el espacio de estados árbol, **verificando** cuando cada nodo **promete o no** retroceder al nodo pariente. Esto es llamado **apodando** el espacio de estados árbol, y el subárbol que consiste de visitas es llamado **espacio de estados árbol apodado**.

● Resumen:

- ▶ *Backtracking* consiste en hacer una **busca** en profundidad en el espacio de estados árbol, **verificando** cuando cada nodo **promete o no** retroceder al nodo pariente. Esto es llamado **apodando** el espacio de estados árbol, y el subárbol que consiste de visitas es llamado **espacio de estados árbol apodado**.

● Algoritmo general:

```
1: void VerificaNodo (nodo  $v$ )
2:   nodo  $u$ ;
3:   if Promete ( $v$ ) then
4:     if Existe una solución en  $v$  then
5:       Escribir la solución
6:     else
7:       for Cada hijo  $u$  de  $v$  do
8:         VerificaNodo ( $u$ )
9:       end for
10:    end if
11:  end if
```

- Algoritmo general: (cont.)

- Algoritmo general: (cont.)
 - ▶ Descripción:

- **Algoritmo general:** (cont.)

- ▶ **Descripción:**

- ★ La raíz del árbol del espacio de estados es pasado a VerificaNodo en el nivel superior.

- Algoritmo general: (cont.)

- ▶ Descripción:

- ★ La raíz del árbol del espacio de estados es pasado a VerificaNodo en el nivel superior.
 - ★ Una visita a un nodo consiste en, primero, verificar si es Prometedor. Si lo es y existe una solución en el nodo, entonces imprimir la solución. Si no existe una solución en un nodo Prometedor, entonces los hijos del nodo son visitados.

- Algoritmo general: (cont.)

- ▶ Descripción:

- ★ La raíz del árbol del espacio de estados es pasado a VerificaNodo en el nivel superior.
 - ★ Una visita a un nodo consiste en, primero, verificar si es Prometedor. Si lo es y existe una solución en el nodo, entonces imprimir la solución. Si no existe una solución en un nodo Prometedor, entonces los hijos del nodo son visitados.
 - ★ La función Promete es diferente en cada aplicación de backtracking.

● Algoritmo general: (cont.)

▶ Descripción:

- ★ La raíz del árbol del espacio de estados es pasado a VerificaNodo en el nivel superior.
- ★ Una visita a un nodo consiste en, primero, verificar si es Prometedor. Si lo es y existe una solución en el nodo, entonces imprimir la solución. Si no existe una solución en un nodo Prometedor, entonces los hijos del nodo son visitados.
- ★ La función Promete es diferente en cada aplicación de backtracking.
- ★ Un algoritmo *backtracking* es idéntico a una búsqueda en profundidad, excepto que los hijos de un nodo son visitados solamente cuando el nodo es prometedor y no existe una solución en el nodo.

● Algoritmo general: (cont.)

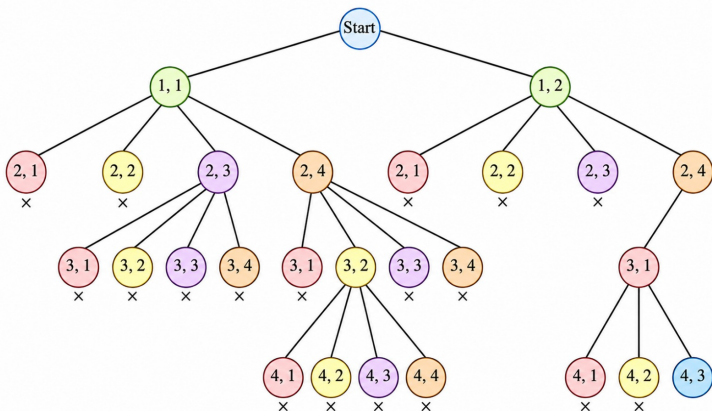
► Descripción:

- ★ La raíz del árbol del espacio de estados es pasado a VerificaNodo en el nivel superior.
- ★ Una visita a un nodo consiste en, primero, verificar si es Prometedor. Si lo es y existe una solución en el nodo, entonces imprimir la solución. Si no existe una solución en un nodo Prometedor, entonces los hijos del nodo son visitados.
- ★ La función Promete es diferente en cada aplicación de backtracking.
- ★ Un algoritmo *backtracking* es identico a una búsqueda en profundidad, excepto que los hijos de un nodo son visitados solamente cuando el nodo es prometedor y no existe una solución en el nodo.
- ★ La función Promete para n-reinas debe retornar falso, sí un nodo y cualquier nodo antecesor colocan las reinas en la misma columna o diagonal.

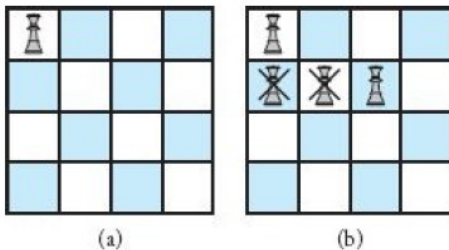
- Algoritmo general: (cont.)

● Algoritmo general: (cont.)

- ▶ Una parte del espacio de estados árbol apodado se produce cuando backtracking soluciona una instancia para el problema de las 4-reinas, donde se muestran los nodos verificados para hallar la primera solución. La solución **esta en el nodo sombreado de color** y **cada nodo no prometedor se marca con una cruz**.

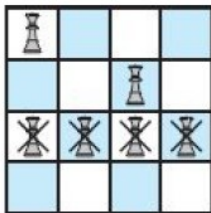


- Algoritmo general: (cont.)

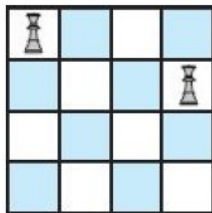


- (a) $\langle 1, 1 \rangle$ es prometedor, pues reina 1 es la primera reina posicionada.
- (b) $\langle 2, 1 \rangle$ no es prometedor, pues reina 1 está en la columna 1
 $\langle 2, 2 \rangle$ no es prometedor, pues reina 1 está sobre la diagonal izquierda.
 $\langle 2, 3 \rangle$ es prometedor.

- Algoritmo general: (cont.)



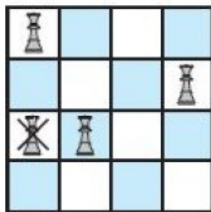
(c)



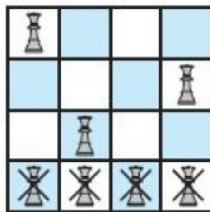
(d)

- (c) $\langle 3, 1 \rangle$ no es prometedor, pues reina 1 es la columna 1
 $\langle 3, 2 \rangle$ no es prometedor, pues reina 2 esta sobre diagonal derecha.
 $\langle 3, 3 \rangle$ no es prometedor, pues reina 2 esta en columna 3
 $\langle 3, 4 \rangle$ no es prometedor, pues reina 2 esta sobre diagonal izquierda
- (d) Retrocediendo a $\langle 1, 1 \rangle$
 $\langle 2, 4 \rangle$ es prometedor.

- Algoritmo general: (cont.)



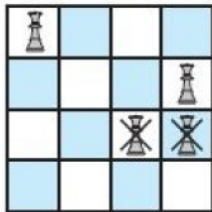
(e)



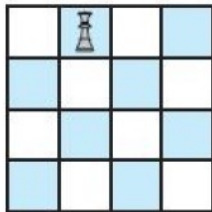
(f)

- (e) $\langle 3, 1 \rangle$ no es prometedor, pues reina 1 esta en columna 1
 $\langle 3, 2 \rangle$ es prometedor
- (f) $\langle 4, 1 \rangle$ no es prometedor, pues reina 1 esta en columna 1
 $\langle 4, 2 \rangle$ no es prometedor, pues reina 3 esta en columna 2
 $\langle 4, 3 \rangle$ no es prometedor, pues reina 3 esta en diagonal izquierda
 $\langle 4, 4 \rangle$ no es prometedor, pues reina 2 esta en columna 4

- Algoritmo general: (cont.)



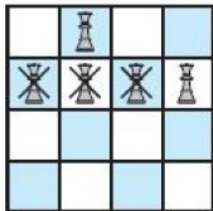
(g)



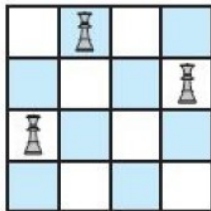
(h)

- (g) Retroceder a $\langle 2, 4 \rangle$.
 $\langle 3, 3 \rangle$ no es prometedor, pues reina 2 esta sobre diagonal derecha.
 $\langle 3, 4 \rangle$ no es prometedor, pues reina 2 esta en columna 4
- (h) Retroceder a raíz.
 $\langle 1, 2 \rangle$ es prometedor.

- Algoritmo general: (cont.)



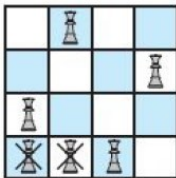
(i)



(j)

- (i) $\langle 2, 1 \rangle$ no es prometedor, pues reina 1 esta sobre diagonal derecha.
 $\langle 2, 2 \rangle$ no es prometedor, pues reina 1 esta en columna 2
 $\langle 2, 3 \rangle$ no es prometedor, pues reina 1 esta sobre diagonal izquierda
 $\langle 2, 4 \rangle$ es prometedor.
- (j) $\langle 3, 1 \rangle$ es prometedor.

- Algoritmo general: (cont.)



(k)

(k) $\langle 4, 1 \rangle$ no es prometedor, pues reina 3 esta en columna 1

$\langle 4, 2 \rangle$ no es prometedor, pues reina 1 esta en columna 2

$\langle 4, 3 \rangle$ es prometedor.

En este punto, (k) se ha encontrado la primera solución, es decir, el nodo sombreado del árbol.

- Algoritmo general: (cont.)

- Algoritmo general: (cont.)
 - ▶ Observaciones:

- Algoritmo general: (cont.)

- ▶ Observaciones:

- ★ Un algoritmo backtracking no necesita crear un árbol. Solamente realiza un seguimiento de los valores en la rama que esta siendo investigada.

- Algoritmo general: (cont.)

- ▶ Observaciones:

- ★ Un algoritmo backtracking no necesita crear un árbol. Solamente realiza un seguimiento de los valores en la rama que esta siendo investigada.
 - ★ El árbol de espacio de estados ² existe implícitamente en el algoritmo, pues en realidad no ha construido.

- Algoritmo general: (cont.)

- ▶ Observaciones:

- ★ Un algoritmo backtracking no necesita crear un árbol. Solamente realiza un seguimiento de los valores en la rama que esta siendo investigada.
 - ★ El árbol de espacio de estados ² existe implícitamente en el algoritmo, pues en realidad no ha construido.
 - ★ El procedimiento VerificarNodo tiene ineficiencia, es decir, verificamos cuando un nodo es prometedor después de pasarlo al procedimiento, esto significa que los registros de activación para los nodos no prometedores son innecesariamente colocados en la pila de registros de activación.